

Checklist

End-to-End Kubernetes Platform Security

1. Identity & Access — Keycloak

- Deploy and configure Keycloak.
- Create a dedicated **Kubernetes realm**.
- Configure an OAuth/OIDC client for the Kubernetes API server.
- Create the required user groups:
 - `platform-admins`
 - `platform-users`
- Create appropriate test users.
- Map Keycloak groups to Kubernetes RBAC groups.
- Configure short-lived access tokens.
- Set token lifetime to approximately **15 minutes**.
- Configure the Kubernetes API server with:
 - OIDC issuer URL
 - Client ID
 - Username claim
 - Groups claim
- Test login through Keycloak.
- Confirm that group membership appears correctly in the issued JWT.

2. Kubernetes RBAC

- Create the application/team namespace.
- Add required governance labels:
 - `team`
 - `environment`
 - `cost-center`
- Create a namespace-scoped developer Role.
- Create RoleBindings for the appropriate Keycloak groups.

- Give platform administrators the required cluster-level troubleshooting permissions.
- Keep normal developers limited to their own namespaces.
- Confirm developers cannot view or modify system namespaces.
- Avoid unnecessary `cluster-admin` access.
- Verify permissions using `kubectl auth can-i`.

3. Secure CI/CD Service Accounts

- Create a separate service account for each application or pipeline.
- Scope each service account to a single namespace.
- Grant only the permissions the pipeline actually requires.
- Avoid `ClusterRoleBinding` to `cluster-admin`.
- Use short-lived projected tokens rather than permanent credentials.
- Ensure CI/CD credentials are not written to build logs.
- Verify the service account can perform permitted deployment operations.
- Verify it cannot access other namespaces.
- Verify it cannot perform unauthorized create/delete operations.

4. OPA / Gatekeeper Policy-as-Code

- Deploy OPA Gatekeeper.
- Configure an approved container registry policy.
- Require CPU limits.
- Require memory limits.
- Prevent privileged containers.
- Prevent root containers where required.
- Enforce namespace labels.
- Configure permitted values or patterns for:
 - Team
 - Environment
 - Cost center
- Configure necessary system namespace exclusions.
- Add required PodDisruptionBudget policy.
- Test a deliberately noncompliant deployment.
- Confirm Gatekeeper rejects unauthorized images.
- Confirm Gatekeeper rejects workloads without resource limits.
- Monitor policy violations.

5. Network Security

- Define ingress traffic rules.
- Define egress traffic rules.
- Create appropriate Kubernetes NetworkPolicies.
- Restrict unnecessary cross-namespace communication.
- Prevent unnecessary host-level network access.
- Configure Istio strict mTLS mode.
- Verify service-to-service traffic is encrypted.
- Confirm unauthorized network paths are blocked.

6. TLS & Certificate Management

- Install/configure `cert-manager`.
- Configure the certificate issuer.
- Integrate certificate management with the Istio Gateway.
- Create the required Certificate resource.
- Store the certificate in the appropriate Kubernetes Secret.
- Configure the application's VirtualService.
- Confirm external traffic uses HTTPS/TLS.
- Verify certificates renew automatically.
- Monitor certificate expiration and renewal state.

7. Validation Testing

- Authenticate as a platform administrator.
- Authenticate as a platform user.
- Authenticate using the CI/CD service account.
- Test authorized actions.
- Test unauthorized actions.
- Confirm namespace boundaries are enforced.
- Confirm RBAC denies permissions that were not explicitly granted.
- Confirm OPA policies reject invalid configurations.
- Confirm network policies block unauthorized traffic.
- Confirm mTLS is active.
- Confirm TLS works for external access.
- Trace a user action from **login** → **token** → **Kubernetes API call**.

8. Audit & Observability

- Enable Kubernetes API audit logging.
- Record:
 - User identity
 - Timestamp
 - Authentication/authorization decisions
 - Secrets access
 - RBAC modifications
 - Privileged operations
 - Request outcomes
- Capture Keycloak login and token events.
- Centralize security-related logs.
- Protect audit logs from modification.
- Apply required log-retention policies.
- Configure alerts for security-sensitive changes.
- Monitor OPA/Gatekeeper policy violations.
- Correlate identity events with Kubernetes activity.

9. Incident Response Drill

- Simulate exposure of a service-account credential.
- Detect the compromised credential.
- Revoke or rotate the credential.
- Verify the compromised credential no longer works.
- Locate the related authentication and API activity in audit logs.
- Determine what resources the credential could access.
- Document the incident timeline.
- Implement preventive measures.
- Confirm preventive controls work during a repeat test.

Final Security Review

- OAuth/OIDC authentication is enabled.
- RBAC authorization follows least privilege.
- Service accounts are narrowly scoped.
- OPA admission policies enforce deployment standards.
- Network policies restrict unnecessary communication.
- mTLS protects service-to-service traffic.
- TLS certificates are automatically managed.
- Security events are centrally observable.
- Audit logs support investigation and compliance.

- No human or service account has unnecessary `cluster-admin` privileges.
- Regular permission reviews, policy updates, and security drills are scheduled.